

Sistema de *comentários II*

Na aula 04, os comentários só podiam nascer. Agora o leitor ganha controle sobre o que escreveu: **editar**, **apagar** e **responder** a outros comentários. Parece simples, mas cada ação esconde uma pergunta de segurança: editar pode mudar o quê? Apagar some com as respostas? Responder pode apontar para outro artigo?

Editar só o texto

"(editado)"

Respostas em um nível

Remover sem quebrar a conversa

Função no banco

Gatilho de proteção

Formação Claude Code 2026 · Baseado em docs.lovable.dev (Connect to Supabase) e supabase.com/docs (Row Level Security), consultados em setembro de 2026 ·
Telas: reprodução ilustrativa

Três ações, *três perguntas*

AÇÃO	QUEM	A PERGUNTA DE SEGURANÇA	A RESPOSTA DESTA AULA
Editar	Só o autor do comentário	A edição pode mudar o artigo, a resposta ou o dono?	Não: um gatilho só deixa mudar o texto
Responder	Qualquer usuário logado	Uma resposta pode apontar para outro artigo, ou para outra resposta?	Não: resposta só a comentário principal do mesmo artigo
Apagar	Só o autor do comentário	E as respostas que já existem?	Sem respostas, apaga; com respostas, remove o texto e mantém a conversa

Moderar comentários **de outras pessoas** (autora e admin) é a aula 06. Hoje, cada um só mexe no que é seu.

Três colunas *novas*

COLUNA	PARA QUÊ
<code>editado_em</code>	Preenchida a cada edição; mostra "(editado)"
<code>resposta_a</code>	Aponta para o comentário principal respondido; vazio = comentário principal
<code>removido_em</code>	Preenchida quando um comentário com respostas é removido
UMA REGRA DE VALOR AJUSTADA Na aula 04, o texto era obrigatório (2 a 1.000). Agora: ou o comentário tem texto válido e não foi removido, ou foi removido e o texto está vazio. Nunca os dois.	

POR QUE UM NÍVEL SÓ DE RESPOSTA

Respostas de respostas viram escadas que, no celular, espremem o texto contra a borda. Um nível mantém a conversa legível; quem quer responder a uma resposta responde ao comentário principal.

`resposta_a` tem **on delete cascade**: se um comentário principal for apagado de verdade (só acontece quando não tem respostas), nada fica órfão.

RLS escolhe linhas, *não colunas*

A política de UPDATE diz **quais comentários** o leitor pode alterar: os dele. Mas não diz **quais colunas**. Sem outra trava, ele poderia editar o próprio comentário e mudar o `artigo_id`, levando-o para outro artigo, até um rascunho.

TRAVA	O QUE GARANTE
Política de UPDATE	Só os próprios comentários; <i>with check</i> mantém o dono
Gatilho de edição	Recusa mudar <code>artigo_id</code> , <code>resposta_a</code> , <code>autor_id</code> e <code>criado_em</code> ; preenche <code>editado_em</code>
Gatilho de resposta (no INSERT)	A resposta aponta para um comentário principal do mesmo artigo

O QUE É UM GATILHO

Um trecho de código que o próprio banco roda **antes** de gravar. Ele confere a mudança e pode recusá-la. Como roda no banco, vale para o app, para a API e para qualquer outra porta.

Esta é uma lição que vale para qualquer app: **RLS protege o acesso às linhas**; o que pode mudar dentro delas é outra regra, e precisa ser escrita.

Apagar *sem quebrar a conversa*



Apagar um comentário com resposta · reprodução ilustrativa

UMA FUNÇÃO NO BANCO DECIDE

`remover_comentario(id)` confere se o comentário é de quem pediu e, então, apaga ou remove. A tela só chama a função e mostra a janela certa.

SITUAÇÃO	O QUE ACONTECE
Comentário sem respostas	Apagado de verdade (DELETE)
Comentário com respostas	Texto apagado, <code>removido_em</code> preenchido; aparece "Comentário removido pelo autor"; as respostas continuam
Resposta	Não tem respostas: sempre apagada de verdade
Comentário de outra pessoa	Recusado pela função e pelas políticas

"Removido" apaga o **texto** no banco, não só na tela. Se o texto ficasse lá, qualquer um ainda o leria pela API, porque a leitura dos comentários é pública.

O pedido *do banco*

Pedido 1 (Plan mode) - banco:

Na tabela comentarios:

1. Novas colunas: `editado_em`, `resposta_a`
(ligada a `comentarios`, on delete cascade)
e `removido_em`.
2. Regra: texto de 2 a 1.000 quando não
removido; texto vazio quando removido.
3. Gatilho no INSERT: `resposta_a` só aponta
para comentário PRINCIPAL do MESMO artigo.
4. Política de UPDATE: só os próprios, com
`with check` mantendo o `autor_id`.
5. Gatilho no UPDATE: recusa mudar `artigo_id`,
`resposta_a`, `autor_id` e `criado_em`; preenche
`editado_em` quando o texto mudar.
6. Função `remover_comentario(id)`: só o autor;
sem respostas, apaga; com respostas, limpa
o texto e preenche `removido_em`.
7. Política de DELETE: só os próprios.

Mostre as migrations antes.

NA MIGRATION, CONFIRA

- ✓ UPDATE com *with check*
- ✓ Os dois gatilhos existem e recusam o que devem
- ✓ A função confere o autor antes de agir
- ✓ Removido = texto vazio no banco
- ✓ Nenhuma permissão para alterar comentários de outros

É a maior migration do módulo até aqui. Leia com calma e, se algo não estiver claro, pergunte no Chat mode antes de aprovar.

O pedido *da interface*

Pedido 2 (Plan mode) - tela:

Na seção de comentários:

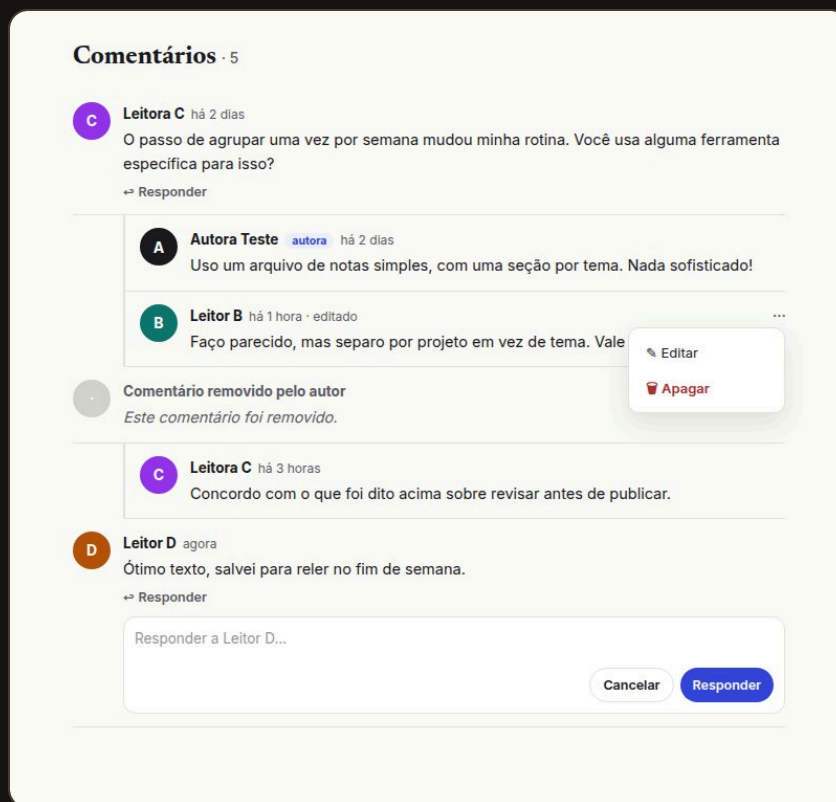
- Nos próprios comentários, menu "..." com Editar e Apagar.
- Editar abre o texto no lugar, com contador, Cancelar e Salvar; depois mostra "editado" junto da data.
- Apagar chama `remove_comentario` e mostra a confirmação certa: com respostas, explica que ficará "Comentário removido pelo autor".
- "Responder" só nos comentários principais; abre um campo logo abaixo; respostas ficam recuadas, um nível só.
- "Comentários · N" não conta os removidos.
- Tudo em tempo real, como na aula 04.

NO PLANO, CONFIRA

- ✓ O menu só aparece nos comentários do próprio leitor
- ✓ Apagar usa a função, não um DELETE direto
- ✓ Não há "Responder" nas respostas
- ✓ A contagem ignora removidos

Esconder o menu nos comentários dos outros é conforto. Quem impede de verdade são as políticas, os gatilhos e a função da página anterior.

A conversa *com respostas*



Leitor B logado · reprodução ilustrativa com dados de teste

- ✓ Respostas **recuadas**, com uma linha à esquerda ligando ao comentário principal
- ✓ Menu ... só no comentário do Leitor B
- ✓ "**editado**" junto da data, para ninguém ser pego de surpresa
- ✓ O removido mantém o lugar, e a resposta dele continua fazendo sentido
- ✓ Campo de resposta abre logo abaixo, com "Responder a Leitor D..."

"Comentários · 5": são 6 itens na conversa, mas o removido não conta.

Editar no lugar, *ler no bolso*

Comentários · 5

B

Leitor B editando

Faço parecido, mas separo por projeto em vez de tema. Vale testar!

68/1000 · só o texto pode ser alterado

Cancelar

Salvar

Editando no lugar · reprodução ilustrativa

- ✓ O texto vira campo no mesmo lugar, sem abrir outra tela
- ✓ "Só o texto pode ser alterado": a regra do gatilho, em linguagem de gente
- ✓ Cancelar desfaz sem salvar nada

A edição conta como mudança no banco. Na aula 09, "editado" não conta como comentário novo nas métricas.

Comentários · 5

C

Leitora C há 2 dias

O passo de agrupar uma vez por semana mudou minha rotina. Você usa alguma ferramenta específica para isso?

↔ Responder

A

Autora Teste autora há 2 dias

Uso um arquivo de notas simples, com uma seção por tema. Nada sofisticado!

B

Leitor B há 1 hora · editado

Faço parecido, mas separo por projeto em vez de tema. Vale testar!

Editar

Apagar

Comentário removido pelo autor

Este comentário foi removido.

C

Leitora C há 3 horas

Concordo com o que foi dito acima sobre revisar antes de publicar.

D

Leitor D agora

Ótimo texto, salvei para reler no fim de semana

390 px · recuo menor

Tentando *burlar as regras*

- 1 Leitor B edita o próprio comentário: aparece "editado".
- 2 No SQL Editor, tente mudar o `artigo_id` de um comentário: o gatilho recusa.
- 3 Tente inserir uma resposta apontando para uma resposta, ou para um comentário de outro artigo: recusado.
- 4 Leitora C apaga um comentário **sem** respostas (some) e um **com** resposta (vira "removido", texto vazio no Table Editor).
- 5 Pergunte no Chat mode se B consegue editar ou apagar o comentário de C: não.

REGISTRO DOS TESTES EXEMPLO

Meu Blog · comentários II

✓ editar o próprio	ok · "editado"
✓ mudar artigo_id (SQL)	recusado
✓ resposta de resposta	recusada
✓ resposta a outro artigo	recusada
✓ apagar sem respostas	apagado
✓ apagar com resposta	removido
✓ texto removido no banco	vazio
✓ B mexe no comentário de C	não
bookmark: "comentários v2"	

O SQL Editor ignora o RLS (você é admin), mas **não ignora gatilhos nem regras de valor**: é o lugar certo para testar as travas das colunas.

Erros comuns *ao editar e responder*

Confiar só no RLS para a edição. Ele escolhe linhas. Sem gatilho, a edição leva o comentário para outro artigo.

"Removido" só na tela. Com leitura pública, o texto continua disponível pela API. Apague no banco.

Apagar em cascata as respostas. Um leitor apaga o próprio comentário e leva junto o que outras pessoas escreveram.

Respostas infinitas. No celular, a quinta resposta tem três palavras por linha. Um nível basta.

Edição silenciosa. Sem "editado", alguém muda o que disse depois de receber resposta e a conversa perde o sentido.

Contar removidos. "Comentários · 6" com um removido engana o leitor.

Teste o que *ficou*

1. O que impede a edição de mudar o artigo de um comentário?

A) A política de UPDATE

B) Um gatilho no banco que só deixa mudar o texto

C) O menu escondido

D) O tempo real

B. RLS escolhe linhas; colunas precisam de outra regra.

2. Um comentário com 2 respostas é apagado pelo autor. O que acontece?

A) Some com as respostas

B) Vira "Comentário removido pelo autor", com texto apagado no banco; as respostas ficam

C) Nada

D) Só some da tela

B. A conversa continua fazendo sentido, e o texto não fica exposto.

3. Por que só um nível de resposta?

A) O Supabase limita

B) Mantém a conversa legível, principalmente no celular

C) Economiza espaço

D) Por causa do RLS

B. Escadas de respostas espremem o texto.

4. Onde testar as travas das colunas?

A) Só pela tela

B) No SQL Editor, que ignora o RLS mas não os gatilhos

C) No Knowledge

D) Não precisa testar

B. Se o gatilho recusa até o admin, recusa qualquer um.

○ RECAPITULANDO

Cada um cuida *do que escreveu*

O leitor agora edita o próprio comentário (só o texto, com "editado"), responde em um nível só a comentários do mesmo artigo e apaga sem quebrar a conversa: sem respostas, some; com respostas, vira "removido", com o texto apagado no banco. Aprendemos que RLS escolhe linhas e que colunas pedem gatilhos, e deixamos a decisão de apagar numa função no servidor. Bookmark "comentários v2".

Gatilho só texto

editado_em

resposta_a um nível

remover_comentario

Texto apagado no banco

Removidos não contam

Próxima aula → Sistema de comentários III

Fontes: docs.lovable.dev (Connect to Supabase: migrations) e supabase.com/docs (Row Level Security), consultados em setembro de 2026. As telas são reproduções ilustrativas feitas para o curso, com contas e comentários de teste.